Startup Funding Prediction and Investment Analytics Using Machine Learning

Saumy Dixit
Registration No. 12404552
saumydixit142@gmail.com
B.Tech, Computer Science and Engineering
School of Computer Science
Lovely Professional University
Phagwara, Punjab, India

***Abstract*—In this paper we take a close look at startup funding in India. Starting from roughly 4,000 cleaned funding rounds from 2015 to 2026, we explore the data and then try regression, classification, clustering and association analysis in Python. Funding amounts are heavily skewed, so we train our models on a log-transformed target. When we sort rounds into low, medium and high funding classes, the Support Vector Machine does best at about 66.5% accuracy, with Logistic Regression close behind at about 65%. For predicting the dollar amount itself, multiple linear regression gives the lowest RMSE, around 5.9×10^7 USD, and K-Means splits the rounds into three groups. The exploratory work shows that activity peaked in 2021–2022, that consumer and fintech startups lead, and that many rounds have no stage on record. Overall, the public attributes we have explain only part of why one round is bigger than another.**

Keywords—startup funding, venture capital, machine learning, regression, classification, K-Means clustering, exploratory data analysis

I. Introduction

Every startup needs money, and every investor has to decide where to put it and how much to commit, usually with very little certainty. India has one of the largest startup ecosystems in the world, and the rounds raised there range from small angel cheques to very large late-stage deals. Looking at these rounds as data can help founders judge what a realistic round looks like, help investors compare sectors and stages, and show policymakers where capital is flowing.

In this work we study a sample of Indian startup funding rounds and ask four simple questions. What does the data look like in terms of sectors, stages, time and size? Can we estimate the dollar amount of a round from its attributes? Can we place a round in the right low, medium or high funding class? And do the rounds fall into natural groups? Everything is done in one Python script built on pandas [2], scikit-learn [1], statsmodels [4] and Matplotlib [3]. The script cleans the data, draws the exploratory figures, trains and compares the models, and saves every result to disk.

Our main contributions are: a repeatable cleaning and feature-preparation pipeline for messy funding data; a side-by-side comparison of three regression and four classification models under the same preprocessing; a K-Means segmentation backed by the elbow method; and a discussion of why the available features give only moderate predictive power.

Work on venture funding commonly uses three kinds of methods: regression to estimate how much a company raises, classification to predict the size or outcome of a round, and clustering to segment markets. We use the same three families, but we test them together on one dataset with identical preprocessing, so any difference in results comes from the models and not from how the data was prepared. The rest of the paper is organised as follows. Section II describes the data and how we prepared it, Section III covers the exploratory analysis, Section IV explains the methods, Section V discusses the results, and Sections VI and VII give the limitations and conclusions.

II. Dataset and Pre-processing

A. Dataset

Our data is a CSV file of Indian startup funding rounds (*india_startup_funding_sample.csv*), where each row is one round. We first tidy the column names into lower-case with underscores and map alternative names to a common schema. The attributes we use are the company name, sector, funding stage, city, announcement date and year, founding year, funding amount in USD (*amount_usd*), number of investors (*investor_count*) and number of news mentions (*news_mentions*).

B. Cleaning

We clean the data in a fixed order so that any run gives the same result. Text columns are trimmed and blank strings become missing values, and duplicate rows are dropped. Numeric columns that arrive as text lose their currency symbols, commas and spaces, placeholders such as “N/A” or “-” become missing values, and the columns are then converted to numbers. We parse the announcement date and use its year to fill any missing *year*. We also create a new feature, *startup_age = year − founded_year*, and set it to missing if it is negative or above 100 years. Rows with a missing or non-positive funding amount, or with no company name, are removed, and infinite values become missing values. About 4,000 rounds remain after cleaning, as the yearly counts in Fig. 3 show. We save the missing-value counts from before and after cleaning so the process can be audited.

C. Feature Preparation

The predictors are sector, stage, city, year, founding year, startup age, investor count and news mentions, limited to those that actually exist in the file. A common scikit-learn *ColumnTransformer* handles them: numeric columns get median imputation and standard scaling, and categorical columns get most-frequent imputation and one-hot encoding, with unseen categories ignored at prediction time. The same preprocessing sits inside every model pipeline, so the imputer and scaler learn from the training data only and nothing from the test set leaks in.

TABLE I
ATTRIBUTES USED IN THE STUDY

Attribute	Type	Role
sector	Categorical	Predictor
stage	Categorical	Predictor
city	Categorical	Predictor
year	Numeric	Predictor
founded_year	Numeric	Predictor
startup_age	Numeric (derived)	Predictor
investor_count	Numeric	Predictor
news_mentions	Numeric	Predictor
amount_usd	Numeric	Target / class source

D. Overall Workflow

The whole analysis runs as one script with ordered stages (Fig. 1). Each stage writes its tables and figures to a *results* folder, including the cleaned dataset, the missing-value counts, the regression and classification score tables, the correlation matrix, the OLS summary, the cluster assignments and the association rules. We fix the random seed at 42, so every split, sample and cluster start can be repeated exactly.

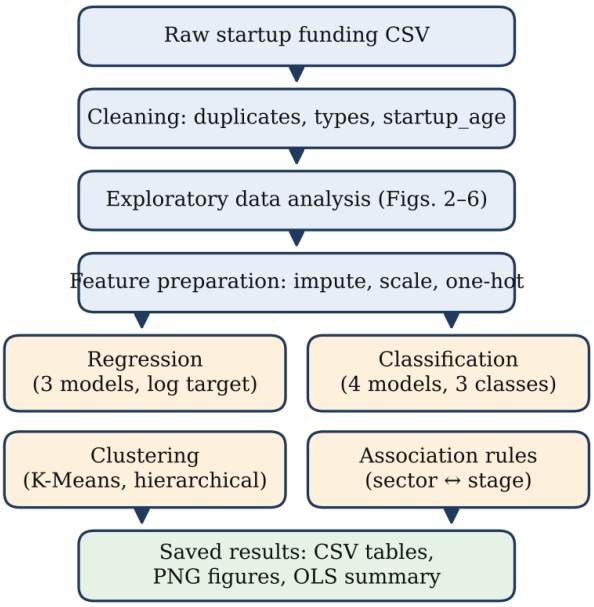


Fig. 1. Workflow of the analysis from raw data to saved results.

III. Exploratory Data Analysis

A. Funding Amounts

Raw funding amounts range over several orders of magnitude, so we look at them on a $\log_{10}$ scale (Fig. 2). On this scale the distribution is roughly bell-shaped, with its peak near $\log_{10} = 6.7$ (about 5 million USD), a lower tail down to about 1,000 USD and an upper tail reaching about 1 billion USD. A few huge rounds therefore dominate the raw dollar scale, which is why we log-transform the regression target in Section IV.

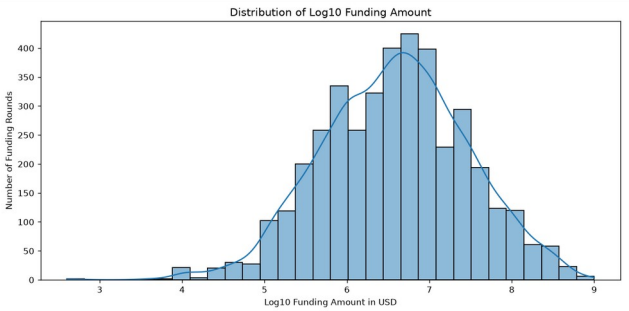


Fig. 2. Distribution of $\log_{10}$ funding amount (USD) with a kernel density estimate.

B. Funding Over Time

Fig. 3 shows the number of recorded rounds each year. It starts at about 240 in 2015, dips to about 190 in 2016, and then climbs steadily to about 320 in 2019–2020. It jumps to a peak of about 540 in 2021, stays near 540 in 2022, and then eases to about 455 in 2023–2024 and about 335 in 2025. The 2026 value of about 113 looks low mainly because the year is incomplete, so it should not be read as a collapse. The rise and fall fit the well-known funding boom of 2021–2022 and the cooling that followed.

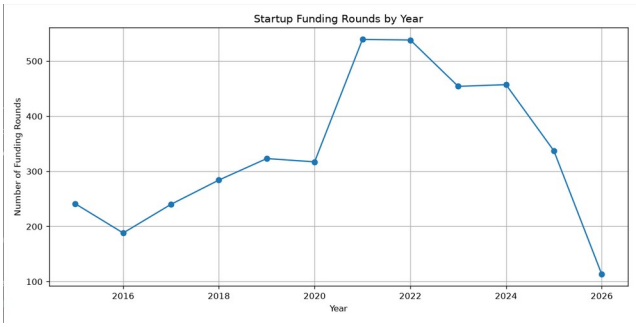


Fig. 3. Number of startup funding rounds per year.

C. Sectors and Stages

Consumer startups have the most rounds (about 700), followed by Fintech (about 460), Retail (about 410) and Food and Agriculture Tech (about 360). Healthtech, Enterprise Applications, Edtech, Financial Services, Food and Agriculture, and Energy Tech make up the rest of the top ten (Fig. 4). Consumer-facing and financial-technology themes are clearly the most visible in this sample.

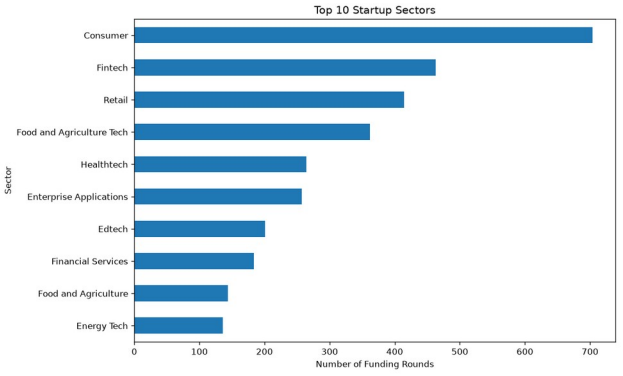


Fig. 4. The ten sectors with the most funding rounds.

Looking at stages (Fig. 5), the biggest group is “Undisclosed” with about 1,200 rounds, so nearly 30% of rounds have no usable stage information. Among the disclosed stages, Seed (about 690), Series A (about 500) and Pre-Series A (about 370) lead, followed by Debt (about 250) and Series B (about 245). The counts then fall steadily: Series

C is about 130, Series D about 80 and Series E about 40. This pyramid shape is typical, since many companies raise early rounds and only a few reach the late stages.

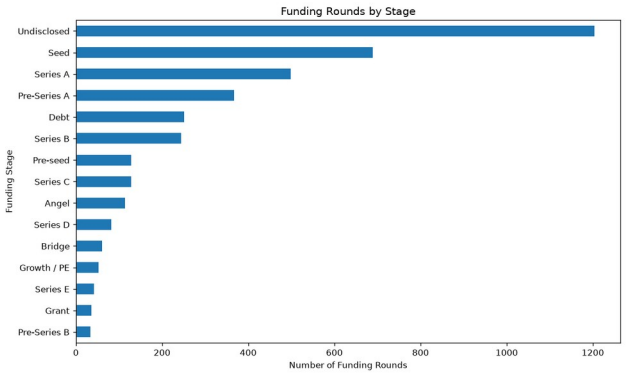


Fig. 5. Number of funding rounds by funding stage.

D. Correlation Analysis

Fig. 6 shows the Pearson correlations between the numeric variables. Funding amount has only weak linear links with the other variables. It rises a little with startup age (0.26) and news mentions (0.14), falls a little with founding year (−0.26), and is almost unrelated to investor count (0.04) and year (−0.01). So older companies tend to raise somewhat larger rounds, but no single numeric variable predicts the amount well. The strongest relationship among the predictors is −0.78 between founding year and startup age, which is expected because startup age is calculated from founding year. News mentions and investor count are moderately correlated (0.44), which suggests that rounds with many investors attract more coverage. The script also runs a Pearson test between investor count and funding amount and reports its *p*-value.

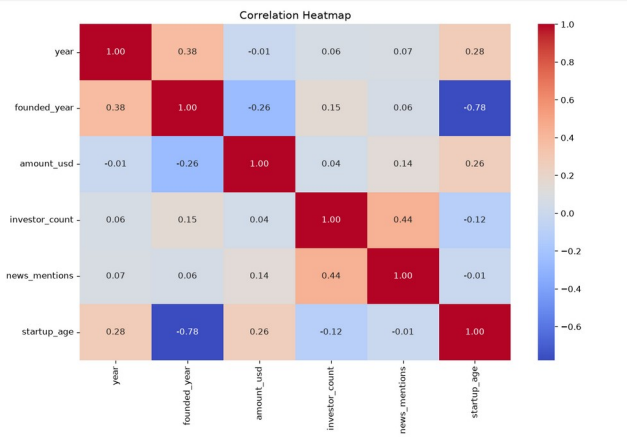


Fig. 6. Correlation heatmap of the numeric variables.

IV. Methodology

A. Regression

Regression is about estimating the funding amount y. Because of the heavy right tail in Fig. 2, we fit the models on a transformed target

$$y' = \ln(1 + y) \tag{1}$$

and convert predictions back with $\hat{y} = \exp(\hat{y}') - 1$, clipped at zero, so that errors are measured in dollars. We split the data 80/20 into training and test sets with a fixed random seed (42) and compare three models. *Simple linear regression* uses a single numeric predictor, the first numeric column in the feature list, which is the funding year. *Multiple linear regression* uses all the prepared features, including the one-hot encoded sector, stage and city. *Polynomial regression* adds a degree-2 expansion of the same single predictor. We score every model with MAE, MSE, RMSE and R^2, where

$$\text{RMSE} = \sqrt{(1/n)\,\Sigma\,(y_i - \hat{y}_i)^2} \tag{2}$$

We also fit an ordinary least squares model on the log amount with statsmodels and save its full summary (coefficients, *p*-values, R^2) to a text file.

B. Classification

For classification we divide the funding amount into three classes using its one-third and two-thirds quantiles: Low (up to the first tercile), Medium (up to the second) and High (above that). This gives three classes of similar size. The split is 80/20 and stratified by class. We evaluate four classifiers, each inside the same preprocessing pipeline: Logistic Regression [5] (up to 2,000 iterations), *k*-Nearest Neighbors [6] (k = 5), a Decision Tree [7] (maximum depth 8) and a Support Vector Machine with an RBF kernel [8]. We also tried Gaussian Naive Bayes, but it needs a dense input matrix and does not run on the sparse one-hot output, so it is left out of the comparison. The metrics are accuracy, weighted precision, recall and F1-score, log-loss and one-vs-rest ROC-AUC:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \tag{3}$$

$$\text{F1} = 2 \cdot P \cdot R / (P + R) \tag{4}$$

where P is precision and R is recall. We also draw the confusion matrix of the most accurate model.

C. Clustering and Association Rules

For the unsupervised part we use only the numeric columns. We remove constant columns, fill missing values with the median, take a random sample of at most 2,000 rows and standardise the values. K-Means [9] is run for k = 2 to 7, and we plot the inertia (within-cluster sum of squares) for the elbow method [10]. We then fit a final model with k = 3, so that the groups line up with the low, medium and high funding classes and stay easy to interpret. We also apply agglomerative hierarchical clustering [11] with single, complete, average and centroid linkage to a sample of 300 rows, cut at three clusters, and save the assignments to compare with K-Means.

To look for links between sectors and stages, we treat every record as two items, “Sector=…” and “Stage=…”. For a rule $A \rightarrow B$ the script computes

$$\text{Support} = n(A \wedge B)/N, \quad \text{Confidence} = n(A \wedge B)/n(A) \tag{5}$$

$$\text{Lift} = \text{Confidence} / \text{Support}(B) \tag{6}$$

following standard association-rule mining [12]. We drop pairs with support below 1%, create rules in both directions and rank them by lift. A lift above 1 means that the sector and stage appear together more often than chance would suggest.

D. Model Configurations

Table II lists the settings of every model. We did not run a hyper-parameter search, so these are simply the values fixed in the script.

TABLE II
MODEL SETTINGS

Model	Settings
Simple linear	One numeric predictor (year)
Multiple linear	All prepared features, one-hot categories
Polynomial	Degree 2, same single predictor
Logistic regression	Maximum 2,000 iterations
k-NN	k = 5 neighbours
Decision tree	Maximum depth 8, seed 42
SVM	RBF kernel, probability on, seed 42
K-Means	k = 3, 10 restarts, 2,000-row sample
Hierarchical	4 linkages, 300-row sample, 3 clusters

V. Results and Discussion

A. Regression Results

Fig. 7 and Table III compare the RMSE of the three regression models on the test set (values read from the chart). Simple linear and polynomial regression perform almost identically, with an RMSE of about 6.45×10^7 USD. That makes sense, because both use the same single predictor, the year, which has a correlation of only −0.01 with the amount (Fig. 6), so a curved fit adds nothing. Multiple linear regression does clearly better at about 5.9×10^7 USD, roughly 8–9% lower. The improvement comes from the categorical information (sector, stage and city) and from the age and investor features.

TABLE III
REGRESSION COMPARISON (TEST SET)

Model	RMSE (USD)	Rank
Simple linear	≈ 6.45×10^7	2 (tie)
Multiple linear	≈ 5.90×10^7	1
Polynomial (deg. 2)	≈ 6.45×10^7	2 (tie)

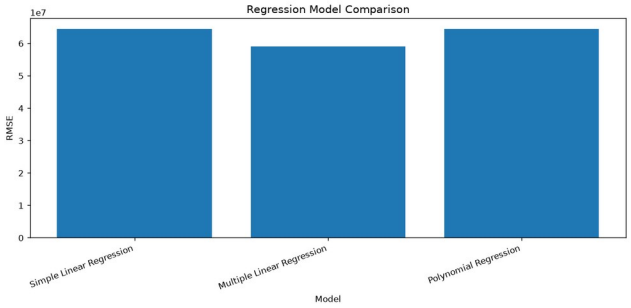


Fig. 7. RMSE of the three regression models on the test set.

Even the best RMSE is large in absolute terms. Because we calculate RMSE after converting back from the log scale, a handful of very large rounds can dominate it. A median-based measure, or evaluation on the log scale, would describe the typical error more fairly. Still, the result shows that a purely tabular model cannot pin down the size of a round tightly. Things that are not in the file, such as the founders' track record, traction, valuation and market conditions, clearly matter too.

B. Classification Results

Fig. 8 compares the accuracy of the four classifiers on the test set. The SVM is best at about 66.5%, Logistic Regression comes next at about 65%, and the Decision Tree and k-Nearest Neighbors reach about 60%. With three roughly balanced classes, random guessing would give about 33%, so every model learns something useful. The ranking is also what we would expect: the linear model and the RBF-kernel SVM cope well with sparse, high-dimensional one-hot input, while k-NN struggles with distances in that space and the depth-limited tree is a weaker fit.

TABLE IV
CLASSIFICATION ACCURACY (TEST SET)

Model	Accuracy	Rank
Support Vector Machine	0.665	1
Logistic Regression	≈ 0.65	2
Decision Tree	≈ 0.60	3
K-Nearest Neighbors	≈ 0.60	4

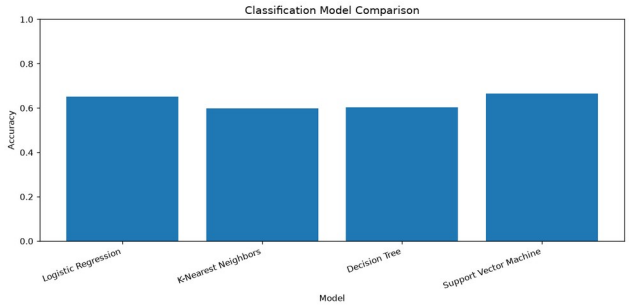


Fig. 8. Accuracy of the four classification models.

The SVM's confusion matrix (Fig. 9) covers 807 test records. The diagonal holds 188 High, 198 Low and 151 Medium correct predictions, which gives 537/807 = 66.5% accuracy. Table V gives the per-class precision and recall worked out from this matrix.

TABLE V
PER-CLASS SVM RESULTS FROM FIG. 9

Class	Precision	Recall	Support
High funding	0.637	0.752	250
Low funding	0.753	0.736	269
Medium funding	0.606	0.524	288

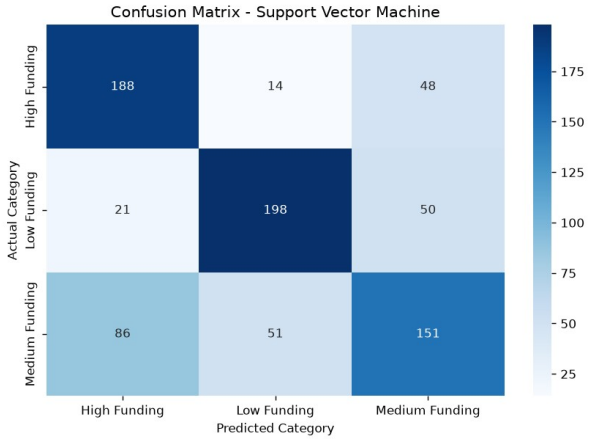


Fig. 9. Confusion matrix of the Support Vector Machine.

The Medium class is the hardest, with a recall of only 52%: 86 Medium rounds are labelled High and 51 are labelled Low. Mix-ups between the two extremes are much rarer (14 High rounds predicted Low and 21 Low rounds predicted High). This is what we would expect from an ordered target. A medium round looks like both of its neighbours, because the class boundaries are quantile cut-offs on a continuous variable, whereas a very small round and a very large one are

clearly different. The model also tends to over-predict High, which has the lower precision of the two extremes.

C. Clustering Results

The elbow plot (Fig. 10) shows inertia falling from about 9,430 at $k = 2$ to about 7,830 at $k = 3$, 6,480 at $k = 4$, 5,430 at $k = 5$, 4,890 at $k = 6$ and 4,530 at $k = 7$. The fall is gradual, and the gain from each extra cluster shrinks from about 1,600 to about 360. There is a mild bend around $k = 5$ but no sharp elbow, which tells us the numeric features do not split into clearly separate groups. We kept three clusters for ease of interpretation, so this is a practical choice and not a statistically sharp optimum.

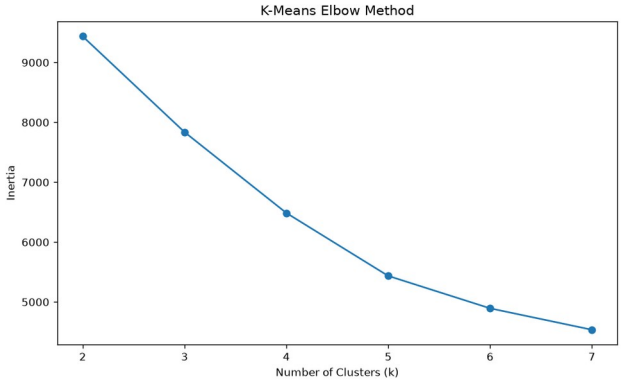


Fig. 10. K-Means elbow plot for $k = 2$ to 7.

Fig. 11 shows the three-cluster result on the first two standardised numeric features. The points sit on a grid because both features are integer-valued, so many records share the same coordinates. The teal cluster fills the lower part of the plot, where the second feature is low, and the yellow cluster takes the upper-left and middle regions. The purple cluster is a small, scattered group that overlaps the others in this view. The clustering uses all the numeric columns while the plot shows only two, so some overlap is natural, and the plot is best read as a rough guide to the segmentation.

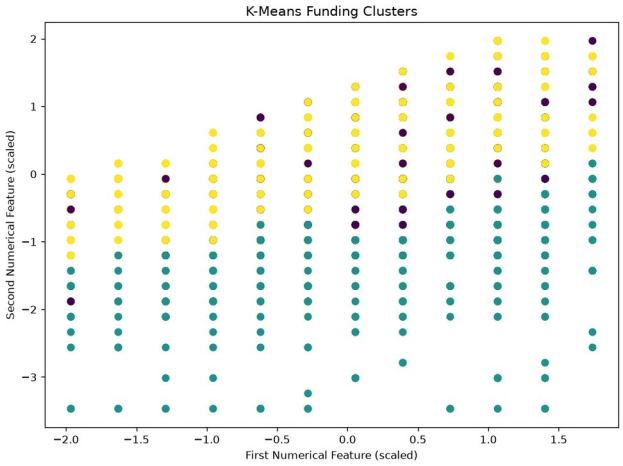


Fig. 11. K-Means clusters ($k = 3$) in the plane of the first two scaled numeric features.

D. Practical Implications

For founders, the amount-estimation results suggest that a benchmark built from sector, stage and company age is a fair starting point, but not a precise forecast. The classification model is more useful in practice: placing a planned round in a low, medium or high band is much easier than naming a dollar figure, and the SVM gets this right about two times in three.

For investors and analysts, the exploratory results show where deal flow is concentrated: consumer, fintech and retail themes, and the early stages from pre-seed to Series A. The sharp drop in recorded rounds after 2022 is a reminder that funding conditions change, so a model trained on boom-period data should be re-checked before it is used in a weaker market. For policymakers, the high share of undisclosed stages is a finding in itself, because it limits how clearly we can see where capital is going.

E. Threats to Validity

Several things limit how far these results can be generalised. The file is described as a sample, so it may not represent all rounds. We removed rounds with no reported amount, which can bias the data towards companies that disclose. The 2026 figures are only partial. We used a single random split, so the reported accuracies carry sampling noise of a few percentage points, and the gap between the SVM and Logistic Regression (about 1.5 points) is too small to call decisive. Finally, all the models use the same predictors, so real improvement has to come from better features and not just from a different algorithm.

VI. Limitations and Future Work

- **Missing stage information.** About 30% of rounds have an undisclosed stage, which weakens an otherwise useful feature. Better imputation or outside data would help.
- **Class thresholds.** Quantile-based classes put one third of the data in each class, even though the real line between a “medium” and a “high” round is a matter of judgement.
- **Model choices.** We fixed the hyper-parameters instead of tuning them and used only one train/test split. Cross-validation and a grid search would give more reliable estimates, and random forests or gradient-boosted trees could be added.
- **Error metric.** RMSE on the dollar scale is sensitive to outliers, so MAE, median error and log-scale metrics should be reported alongside it.
- **Further analyses.** The hierarchical-clustering and association-rule outputs are saved as CSV files and deserve a closer look, for example by profiling each cluster by sector and stage.
- **Clustering quality.** Silhouette scores should back up the elbow plot, and categorical features could be brought in through suitable distance measures.

VII. Conclusion

In this paper we built a complete analytics pipeline for Indian startup funding data, from cleaning through to modelling. The exploratory analysis showed a peak in activity in 2021–2022, a focus on consumer and fintech startups, and a large share of rounds with no stage on record. Funding amounts are strongly skewed, which justifies the log transform, and their linear correlations with other variables are weak. For regression, multiple linear regression had the lowest error (RMSE about 5.9×10^7 USD), while simple and polynomial regression on year alone were no better than each other. For sorting rounds into low, medium and high funding,

the SVM was the most accurate (66.5%), and most of its mistakes fell on the Medium class. K-Means with three clusters gave a workable but not sharply defined segmentation. Overall, tabular attributes capture a meaningful but limited part of what decides the size of a round. Richer features, tuned models and more careful validation are the natural next steps. In particular, adding company-level information such as valuation, revenue, team background and investor quality, and validating with repeated cross-validation, should give the largest gains.

REFERENCES

[1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[3] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[4] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 92–96.

[5] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.

[6] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[7] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[8] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Stat. Probab.*, vol. 1, 1967, pp. 281–297.

[10] R. L. Thorndike, "Who belongs in the family?" *Psychometrika*, vol. 18, no. 4, pp. 267–276, 1953.

[11] F. Murtagh and P. Contreras, "Algorithms for hierarchical clustering: An overview," *WIREs Data Min. Knowl. Discov.*, vol. 2, no. 1, pp. 86–97, 2012.

[12] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1993, pp. 207–216.